Manual de Ingeniería: Síntesis, Despliegue y Operación Sistema de Control de Acceso Autónomo (Yocto 6.0)

Ingeniería de Sistemas Integrados

Octubre 2026

Índice

1. Requisitos Previos y Entorno Host **2**
1.1. Instalación de Dependencias del Host 2

2. Estructuración del Proyecto Yocto **2**
2.1. Inicialización del Entorno 2
2.2. Activación de BitBake 2
2.3. Inclusión de Capas (`bblayers.conf`) 3

3. Configuración Maestra del Hardware (`local.conf`) **3**

4. Inyección de Redundancia de Red (Ethernet y Wi-Fi) **4**
4.1. Configuración de Credenciales 4

5. Síntesis (Compilación) y Flasheo **4**
5.1. Construcción de la Imagen (`bitbake`) 4
5.2. Escritura Segura a la MicroSD 4

6. Puesta en Marcha de Hardware (Despliegue Zero-Touch) **4**
6.1. Ensamblaje del Circuito Físico 5
6.2. Auto-Descubrimiento de IP (Técnica Sneakernet) 5

7. Operación Remota y Enlace de Video **5**
7.1. Inyección de Dirección Dinámica 5
7.2. Ejecución del Receptor de Monitoreo (Windows) 6

1. Requisitos Previos y Entorno Host

Para garantizar una compilación exitosa sin errores de dependencias, el sistema anfitrión (Host) debe ser una distribución Linux basada en Debian (recomendado Ubuntu 22.04 LTS o 24.04 LTS) con al menos 60 GB de espacio libre en disco duro y 8 GB de memoria RAM.

1.1. Instalación de Dependencias del Host

Abra una terminal en el entorno Ubuntu y ejecute el siguiente comando para instalar las herramientas de compilación cruzada, utilidades de red y el software de flasheo de imágenes (`bmaptool`):

```
sudo apt update
sudo apt install -y gawk wget git diffstat unzip texinfo gcc build-
   essential chrpath socat cpio python3 python3-pip python3-pexpect xz-
   utils debianutils iputils-ping python3-git python3-jinja2 libegl1-mesa
   libsdl1.2-dev pylint3 xterm python3-subunit mesa-common-dev zstd liblz4
   -tool bmap-tools
```

2. Estructuración del Proyecto Yocto

El sistema operativo se construye a través del sistema Yocto Project (rama *Wrynose*). Se debe crear un directorio de trabajo y clonar las capas fundamentales.

2.1. Inicialización del Entorno

El proyecto no utiliza la distribución de referencia empacada (Poky), sino que está estructurado directamente sobre los repositorios modulares de OpenEmbedded. Cree la estructura del proyecto y descargue las capas (*layers*) requeridas. Asumiendo que el proyecto reside en `~/taller-yocto`:

```
mkdir -p ~/taller-yocto/layers
cd ~/taller-yocto/layers

# Clonar repositorios modulares (Rama wrynose / compatible)
git clone -b wrynose git://git.openembedded.org/openembedded-core
git clone -b 2.8 git://git.openembedded.org/bitbake
git clone -b wrynose git://git.openembedded.org/meta-openembedded
git clone -b wrynose git://git.yoctoproject.org/meta-raspberrypi

# Clonar capa personalizada del proyecto (Aplicacion y configuraciones)
git clone https://github.com/sau1781/P1_Em_Control-Acceso.git meta-control
   -acceso
```

2.2. Activación de BitBake

Inicialice el entorno de construcción. Este script creará el directorio `build` y activará los comandos de BitBake:

```
cd ~/taller-yocto
source layers/poky/oe-init-build-env buil
```

2.3. Inclusión de Capas (`bblayers.conf`)

Verifique que el archivo `conf/bblayers.conf` incluya las capas recién descargadas. Añada las siguientes rutas absolutas:

```
BBLAYERS ?= "
/home/usuario/taller-yocto/layers/poky/meta
/home/usuario/taller-yocto/layers/poky/meta-poky
/home/usuario/taller-yocto/layers/poky/meta-yocto-bsp
/home/usuario/taller-yocto/layers/meta-openembedded/meta-oe
/home/usuario/taller-yocto/layers/meta-openembedded/meta-python
/home/usuario/taller-yocto/layers/meta-openembedded/meta-networking
/home/usuario/taller-yocto/layers/meta-raspberrypi
/home/usuario/taller-yocto/layers/meta-control-acceso
"
```

3. Configuración Maestra del Hardware (`local.conf`)

El archivo `local.conf` dicta las directrices globales de la imagen. Desde la carpeta `build`, edite el archivo `conf/local.conf` y asegúrese de definir los siguientes parámetros al final del documento:

```
1. Definicion de la M quina (Hardware destino)
MACHINE ?= "raspberrypi4-64"
2. Gestor de arranque y licenciamiento
INIT_MANAGER = "systemd"
LICENSE_FLAGS_ACCEPTED += "synaptics-killswitch"
3. Permisos y Seguridad SSH (Bypass de PAM)
EXTRA_IMAGE_FEATURES += "allow-empty-password empty-root-password allow-
   root-login ssh-server-openssh"
4. Inyeccion de Dependencias en la Imagen Final
IMAGE_INSTALL:append = " control-acceso procps userland systemd-networkd
   wpa-supplicant linux-firmware-rpidistro-bcm43455 python3-html nano"
5. Configuracion del Bootloader (GPIO y UART)
RPI_EXTRA_CONFIG = ' \n
enable_uart=1 \n
gpio=27=op,dl \n
gpio=22=op,dh \n
'
```

Nota de Ingeniería: La configuración `gpio=27=op,dl` fuerza a nivel de hardware que el relé permanezca apagado (estado lógico bajo) durante el inicio, previniendo aperturas accidentales antes de que la aplicación tome el control.

4. Inyección de Redundancia de Red (Ethernet y Wi-Fi)

Para operar de manera autónoma, el dispositivo debe ser capaz de negociar una IP por cable o de conectarse a un Hotspot celular sin interacción humana.

4.1. Configuración de Credenciales

Si ha clonado el repositorio `meta-control-acceso`, localice el archivo de credenciales de Wi-Fi en:

`recipes-core/systemd/systemd-conf/wpa_supplicant-wlan0.conf`

Modifíquelo insertando el SSID y contraseña de su punto de acceso celular. **Precaución:** Se prohíbe el uso de espacios o caracteres especiales (ej. tildes) en el nombre de la red para asegurar compatibilidad estricta con el demonio de Linux. `recipes-core/systemd/systemd-conf/wpa_suppli`
Modifíquelo insertando el SSID y contraseña de su punto de acceso celular. **Precaución:** Se prohíbe el uso de espacios o caracteres especiales (ej. tildes) en el nombre de la red para asegurar compatibilidad estricta con el demonio de Linux.

```
network={
ssid="MiRedCelular"
psk="Password123"
}
```

5. Síntesis (Compilación) y Flasheo

5.1. Construcción de la Imagen (`bitbake`)

Con el entorno configurado, proceda a compilar el sistema operativo base. Este proceso consolidará el *Kernel*, el *RootFS*, las recetas de Python, OpenCV y GStreamer.

```
bitbake core-image-base
```

5.2. Escritura Segura a la MicroSD

Una vez finalizada la tarea, el binario resultante se ubicará en la carpeta de despliegue. Inserte la MicroSD (mínimo 16GB) en el computador Host y ejecute `lsblk` para identificar su montaje (ej. `/dev/sda`). Para evitar condiciones de carrera (*Race Conditions*) con el entorno gráfico de Ubuntu, encadene el comando de desmontaje con la herramienta `bmaptool`:

```
sudo umount /dev/sda* 2>/dev/null ; sudo bmaptool copy tmp/deploy/images/
   raspberrypi4-64/core-image-base-raspberrypi4-64.rootfs.wic.bz2 /dev/sda
```

Espere a que la consola reporte "bmaptool: copy complete". La memoria está lista para producción.

6. Puesta en Marcha de Hardware (Despliegue Zero-Touch)

El sistema ha sido diseñado para arrancar sin monitor ni teclado (*Headless*).

6.1. Ensamblaje del Circuito Físico

Proceda a cablear los periféricos en la Raspberry Pi 4 de acuerdo a la siguiente especificación de pines físicos:

- **Cámara Web:** Conectar a cualquier puerto USB 3.0 (Azul).
- **LED Verde (Apertura):** Ánodo al **Pin Físico 13**.
- **LED Rojo (Bloqueo):** Ánodo al **Pin Físico 15**.
- **Botón (Salida Manual):** Conectar un terminal al **Pin Físico 11**.
- **GND (Tierra Lógica):** Conectar las resistencias catódicas de los LEDs y el segundo terminal del botón al **Pin Físico 14**.

6.2. Auto-Descubrimiento de IP (Técnica Sneakernet)

1. Encienda su Hotspot móvil (o conecte el cable Ethernet a un enrutador DHCP).
2. Conecte el cable de alimentación (USB-C) a la Raspberry Pi 4.
3. Espere exactamente **60 segundos**. Durante este tiempo, la placa negociará su red y ejecutará un script que escribe la IP asignada directamente en la tarjeta MicroSD.
4. Si no puede visualizar la IP asignada en la pantalla de su celular/enrutador, desconecte la Raspberry Pi, extraiga la MicroSD y conéctela a su computador.
5. Abra la partición FAT32 visible (`boot`) y abra el archivo de texto $\texttt{ip}_secreta.txt.AllconstarlaIPasigna$

7. Operación Remota y Enlace de Video

Conociendo la IP de la Raspberry Pi, y asegurándose de que la computadora receptora (Windows) esté conectada al mismo Hotspot o red local, establezca un túnel seguro.

7.1. Inyección de Dirección Dinámica

Conéctese a la placa vía SSH utilizando la llave pública inyectada previamente:

```
ssh -i ~/.ssh/llave_pi root@ <IP_RASPBERRY_PI>
```

Una vez autenticado en la placa, edite el archivo de entorno volátil para informarle a GStreamer a qué dirección IP enviar el video. Sustituya la IP por la dirección IPv4 actual de su computadora Windows (obtenida mediante `ipconfig`).

```
nano /etc/default/control-acceso
```

```
ACCESO_FUENTE=webcam
ACCESO_HOST=<IP_DE_SU_COMPUTADORA_WINDOWS>
```

Guarde el archivo y reinicie el supervisor de sistema para que adopte la nueva dirección:

```
systemctl restart control-acceso.service
```

7.2. Ejecución del Receptor de Monitoreo (Windows)

Inmediatamente después de reiniciar el servicio, ejecute el *Pipeline* de recepción en la consola CMD de Windows. El comando a continuación inicializa el receptor RTP/UDP por el puerto 5000, aplica correctores de tiempo, decodifica el bloque H.264 por *software* y genera un sumidero visual asíncrono.

```
gst-launch-1.0 -v udpsrc port=5000 ! "application/x-rtp, media=video,
   clock-rate=90000, encoding-name=H264, payload=96" ! rtph264depay !
   h264parse ! avdec_h264 ! videoconvert ! autovideosink sync=false
```

Si los pasos se siguieron al pie de la letra, el monitor de GStreamer desplegará la transmisión en vivo, y el sistema de Control de Acceso estará oficialmente operativo, registrando eventos en `/home/root/control-acceso/registro.csv`.